Take-Home Assignment: LLM-Based Open-Source Upgrade Risk Extraction

Background

Software upgrades often introduce breaking changes, deprecated APIs, changed defaults, removed configuration options, runtime requirements, dependency constraints, or behavior changes.

These risks are usually described across many public sources: release notes, migration guides, changelogs, GitHub issues, pull requests, documentation pages, upgrade guides, package metadata, and community discussions.

Your assignment is to build an LLM-assisted workflow that gathers and processes public information about open-source software upgrades, then extracts a structured set of **upgrade risk paths**.

An upgrade risk path is a set of conditions under which an upgrade may break an application, require migration work, or create operational risk. For example, an upgrade may be risky if:

- The application uses an API removed in the target version.
- The target version changes a default behavior.
- The application depends on a package version that is no longer supported.
- The target version requires a newer runtime.
- A configuration option was renamed, removed, or changed.
- A database migration or data format change is required.
- A feature is still supported but behaves differently under specific conditions.

This assignment focuses only on **risk extraction from public documentation**. You do **not** need to analyze a real codebase, run the software, or perform the upgrade.

Time Expectation

Please spend approximately **4 hours** on this assignment.

We are not looking for a production-ready system. We are looking for evidence that you can design an LLM workflow, gather public information, process noisy documentation, extract structured facts, reason carefully about uncertainty, and produce evidence-grounded outputs.

Input Upgrades

Please analyze the following open-source software upgrades:

1. **Django**: `3.2` → `4.2`
2. **React**: `17` → `18`
3. **Kubernetes**: `1.23` → `1.29`
4. **PostgreSQL**: `13` → `16`
5. **Next.js**: `12` → `14`

Please submit outputs for **all 5 upgrades**.

These upgrades were selected because they have public documentation and require more than simply checking whether a newer version exists.

Allowed Public Sources

Your workflow should gather and process public information from relevant sources.

At minimum, use several of the following source types where available:

- Official release notes.
- Official migration or upgrade guides.
- Official changelogs.
- Official API documentation.
- GitHub releases.
- GitHub issues or pull requests.
- Deprecation notices.
- Package registry metadata.
- Maintainer blog posts.
- Reputable community writeups, only as supporting context.

Prefer official project sources when available. Third-party sources may be useful for discovery or clarification, but they should not override official documentation.

Goal

Build an LLM-based workflow that takes an open-source project, current version, and target version as input and outputs a structured upgrade risk model.

Each risk model should contain one or more **upgrade risk paths**.

Each risk path should describe a specific way the upgrade could create breakage, migration work, or operational risk.

For each risk condition, your system should include:

- A concise condition statement.
- The evidence supporting the condition.
- The source URL or document reference.
- Whether the condition is required, optional, alternative, disqualifying, or uncertain.
- The reasoning used to derive it.
- Any ambiguity or disagreement across sources.
- A suggested migration or validation action, when available.

The output should be useful to a downstream system that later evaluates whether a specific codebase or deployment is affected by the risk.

Example Concept

For a hypothetical upgrade, a weak output would be:

```
{
  "project": "ExampleFramework",
  "upgrade": "1.0_to_2.0",
  "risks": [
    "There are breaking changes",
    "Some APIs changed",
    "The app may need migration"
  ]
}
```

A stronger output would be:

```
{
  "project": "ExampleFramework",
  "current_version": "1.0",
  "target_version": "2.0",
  "risk_paths": [
    {
      "path_id": "removed_config_option_path",
      "path_name": "Removed configuration option risk",
      "description":
"Applications using the removed cache.backend configuration option may fail or
behave differently after upgrading.",
      "risk_level": "high",
      "conditions": [
        {
          "condition_id": "target_version_removes_cache_backend",
```

```
          "statement": "Version 2.0 removes support for the cache.backend
configuration option.",
          "category": "removed_configuration",
          "required": true,
          "operator": "present",
          "expected_value": "cache.backend is removed in target version",
          "evidence": [
            {
              "source_id": "source_001",
              "source_type": "official_migration_guide",
              "quote_or_summary":
"The migration guide states that cache.backend was removed in version 2.0 and
replaced by cache.provider.",
              "relevance": "This establishes that the upgrade introduces a
breaking configuration change."
            }
          ],
          "confidence": "high",
          "ambiguities": []
        },
        {
          "condition_id": "application_uses_removed_config",
          "statement": "The application uses the removed cache.backend
configuration option.",
          "category": "application_usage",
          "required": true,
          "operator": "present",
          "expected_value": "cache.backend",
          "evidence": [
            {
              "source_id": "source_001",
              "source_type": "official_migration_guide",
              "quote_or_summary": "The guide instructs users to replace
cache.backend with cache.provider before upgrading.",
              "relevance": "This implies applications using the old option
require migration."
            }
          ],
          "confidence": "high",
          "ambiguities": []
        }
      ],
      "suggested_actions": [
        "Search the application configuration for cache.backend.",
        "Replace cache.backend with cache.provider before upgrading.",
        "Add a test that validates cache initialization after upgrade."
      ]
    }
```

```
  ]
}
```

Requirements

Part 1: Public Source Gathering

Build a workflow that, given a project name, current version, and target version, collects relevant public information.

Your implementation may use search APIs, manual seed URLs, static downloaded documents, or a combination of these.

For each upgrade, produce a source inventory:

```
{
  "project": "Django",
  "current_version": "3.2",
  "target_version": "4.2",
  "sources": [
    {
      "source_id": "source_001",
      "title": "Example Official Migration Guide",
      "url": "https://...",
      "source_type": "official_migration_guide",
      "retrieved_at": "2026-06-26T00:00:00Z",
      "why_relevant": "Documents backwards-incompatible changes between the
current and target versions."
    }
  ]
}
```

Your source inventory should show that the workflow made reasonable choices about which documents to trust.

Part 2: Document Processing

Process the collected documents into an intermediate representation that is useful for extraction.

This can be simple. For example:

- Chunk documents into sections.

- Extract metadata.
- Classify sources by type.
- Deduplicate repeated information.
- Identify candidate sections related to breaking changes, deprecations, removed APIs, changed defaults, runtime requirements, dependency requirements, configuration changes, migration steps, and known upgrade issues.

You may use embeddings, keyword search, LLM-based extraction, deterministic parsing, or a combination of techniques.

Please explain your approach.

Part 3: Upgrade Risk Path Extraction

For each upgrade, extract one or more risk paths.

Each risk path should include:

```
{
  "path_id": "string",
  "path_name": "string",
  "risk_type": "removed_api | changed_behavior | changed_default |
dependency_requirement | runtime_requirement | configuration_change |
data_migration | operational_risk | unknown",
  "risk_level": "critical | high | medium | low | unknown",
  "affected_surface":
"application_code | configuration | dependencies | runtime | database |
deployment | tests | unknown",
  "description": "string",
  "conditions": [],
  "suggested_actions": []
}
```

Each condition should include:

```
{
  "condition_id": "string",
  "statement": "string",
  "category": "version | api_usage | configuration | dependency | runtime |
deployment | database | behavior | test_coverage | unknown",
  "required": true,
  "operator": "equals | not_equals | in_range | less_than | greater_than |
contains | absent | present | unknown",
  "expected_value": "string or structured value",
```

```
  "evidence": [
    {
      "source_id": "source_001",
      "source_type": "official_migration_guide",
      "quote_or_summary": "short evidence snippet or paraphrase",
      "relevance": "why this evidence supports the condition"
    }
  ],
  "confidence": "high | medium | low",
  "ambiguities": []
}
```

A good extraction should distinguish between:

- Required risk conditions.
- Alternative risk paths.
- Removed APIs or features.
- Changed defaults or behavior changes.
- Runtime and dependency requirements.
- Configuration changes.
- Migration steps.
- Known incompatibilities.
- Conditions that are suspected but not clearly supported.
- Contextual facts that are useful but not actual risk conditions.

Part 4: Evidence and Grounding

Every extracted condition must be grounded in evidence.

Your system should avoid producing unsupported conditions. If the LLM is unsure, it should mark the condition as uncertain rather than inventing details.

For each upgrade, include a short evidence summary:

```
{
  "project": "Django",
  "upgrade": "3.2_to_4.2",
  "evidence_summary": {
    "high_confidence_risks": [],
    "medium_confidence_risks": [],
    "low_confidence_or_disputed_risks": [],
    "sources_with_disagreements": [],
    "open_questions": []
```

```
  }
}
```

We are especially interested in how your workflow handles conflicting or incomplete public information.

Part 5: Final Output

Produce a final structured output for each required upgrade.

The output should be machine-readable JSON.

Example shape:

```
{
  "project": "Django",
  "current_version": "3.2",
  "target_version": "4.2",
  "summary": "Short summary of the upgrade and major risk categories.",
  "source_inventory": [],
  "risk_paths": [
    {
      "path_id": "string",
      "path_name": "string",
      "risk_type": "string",
      "risk_level": "string",
      "affected_surface": "string",
      "description": "string",
      "conditions": [],
      "suggested_actions": []
    }
  ],
  "open_questions": [],
  "overall_confidence": "high | medium | low"
}
```

Implementation Expectations

You may use any programming language.

You should use an LLM somewhere in the workflow. This could be for:

- Source triage.

- Document summarization.
- Candidate risk extraction.
- Risk path construction.
- Evidence selection.
- Consistency checking.
- Output normalization.

However, the workflow should not be a single unstructured prompt. We want to see how you decompose the task.

A strong submission will likely include:

1. A pipeline or agentic workflow.
2. Clear prompts or prompt templates.
3. A structured schema for risk paths and conditions.
4. Evidence-grounding and citation handling.
5. A validation or self-checking step.
6. Example outputs for all 5 upgrades.
7. A short design note explaining tradeoffs.

Required Design Note

Include a short design note, approximately 1–2 pages, answering:

1. How did you decompose the extraction problem into workflow stages or agents?
2. What role did the LLM play?
3. What parts of the pipeline are deterministic?
4. How did you prevent hallucinated upgrade risks?
5. How did you decide whether something is a real risk condition versus useful context?
6. How did you handle conflicting or incomplete public documentation?
7. How would you scale this from these upgrades to thousands of projects and versions?
8. How would you evaluate precision and recall of extracted risks?
9. What would you improve with more time?
10. How would this output be consumed by a downstream upgrade assessment engine?

Validation Requirements

Include at least one validation mechanism.

Examples:

- JSON schema validation.
- A second-pass LLM critic that checks whether every risk condition has evidence.
- A deterministic check that rejects conditions without source references.

- A confidence scoring rubric.
- A comparison between extracted risks and manually written expectations.
- A test that verifies each risk path has at least one target-version condition and at least one application/deployment condition when applicable.

You do not need to build a perfect evaluator. We want to see that you are thinking about reliability.

Deliverables

Please submit a GitHub repository, zip file, or shared folder containing:

1. `README.md`
2. How to run your solution.
3. Setup instructions.
4. Required API keys or how to run with cached/static sources.
5. Assumptions and limitations.
6. Source code
7. Pipeline implementation.
8. Prompts or agent definitions.
9. Data schemas.
10. Any retrieval or processing logic.
11. Sample outputs
12. Final JSON outputs for all 5 upgrades.
13. Source inventories for all 5 upgrades.
14. Intermediate extraction artifacts if available.
15. Design note
16. Answers to the required design questions.
17. Optional but encouraged
18. Tests.
19. JSON schema.

20. Cached source documents.
21. CLI, for example:

```
python extract_upgrade_risks.py --project Django --current-version 3.2 --target-version 4.2
```

Constraints

Please do not:

- Analyze private repositories.
- Process proprietary source code.
- Run production upgrades.
- Submit issues or comments to open-source projects as part of the assignment.
- Present the output as a complete migration plan for a real production system.

Focus only on extracting upgrade risk conditions from public written documentation.

Evaluation Criteria

We will evaluate submissions on the following dimensions:

- **Risk extraction quality:** Extracts specific, checkable upgrade risks rather than vague migration summaries.
- **Path modeling:** Separates alternative risk paths, required conditions, migration actions, and contextual facts.
- **Evidence grounding:** Links every risk condition to relevant source evidence and avoids unsupported claims.
- **LLM workflow design:** Uses LLMs purposefully within a decomposed workflow rather than a single unstructured prompt.
- **Public-source handling:** Selects, ranks, deduplicates, and processes public sources thoughtfully.
- **Uncertainty handling:** Marks ambiguity, incomplete evidence, source disagreement, and confidence levels clearly.
- **Validation:** Includes a mechanism to detect unsupported, malformed, or low-confidence outputs.
- **Engineering quality:** Provides readable, runnable, maintainable code with clear setup instructions.
- **Output usability:** Produces structured JSON consumable by a downstream upgrade assessment engine.
- **Product judgment:** Makes practical tradeoffs and explains limitations without overengineering.

Bonus Areas

These are optional.

1. Add one or more additional open-source upgrades beyond the required 5.
2. Add a human-review interface or markdown report.
3. Add a confidence scoring rubric.
4. Add source ranking based on authority and specificity.
5. Add deduplication across sources.
6. Add an LLM judge that flags unsupported risks.
7. Add a graph representation of upgrade risk paths.
8. Add a small manually curated gold set for one upgrade and compare model output against it.
9. Add support for extracting concrete code search queries for each risk condition.
10. Add support for multiple LLM providers or a local model fallback.

What We Are Looking For

Domain expertise in any specific open-source project is not required.

We are looking for someone who can:

- Build reliable LLM workflows over messy real-world information.
- Convert unstructured public documentation into structured decision logic.
- Design evidence-grounded extraction systems.
- Think clearly about uncertainty and verification.
- Separate facts, assumptions, and unsupported claims.
- Build maintainable systems rather than one-off demos.

Please include notes about what you trusted, what you did not trust, and where your workflow may fail.